

Projektarbeit: Pawsitters - A Pet Holiday Platform

Ziel der Projektarbeit

Im Rahmen dieser Veranstaltung entwickeln Sie in Gruppen eine Softwarelösung in Java: eine „**Pawsitters: Pet Holiday Platform**“. Ziel ist es, Tierhalter, die während eines Urlaubs eine Betreuung für ihr Haustier suchen, mit Personen zu verbinden, die bereit sind, Tiere gegen Bezahlung aufzunehmen.

Die Anwendung soll es Tierhaltern ermöglichen, ein Profil anzulegen, ihr Haustier zu registrieren und einen Zeitraum für ihre Abwesenheit zu definieren. Gastgeber können eigene Profile erstellen, angeben, welche Tierarten sie aufnehmen können, ihre Verfügbarkeit festlegen und einen Preis pro Woche definieren. Gastgeber können Angebote an Tierhalter senden. Tierhalter können diese Angebote einsehen, eines auswählen und andere ablehnen.

Neben der funktionalen Umsetzung liegt der Fokus auf **Softwarearchitektur, Testing, Dokumentation, Security-Verständnis, Einsatz moderner Werkzeuge sowie Teamarbeit**.

Rahmenbedingungen

Die Umsetzung erfolgt in **Gruppen von drei Studierenden**. Jede Gruppe arbeitet in einem eigenen Repository (GitLab oder alternativ GitHub).

Sie müssen mich als **Contributor (Mitwirkenden)** zu Ihrem Repository hinzufügen, damit ich Ihre Fortschritte nachvollziehen kann (*Ana Nicolaescu - ana.nicolaescu@heilbronn.dhbw.de*).

Alle Gruppenmitglieder müssen aktiv und nachvollziehbar zum Projekt beitragen.

Technologische Anforderungen

Die Anwendung muss in **Java** umgesetzt werden. Es wird empfohlen, **Spring Boot** in Kombination mit **Maven** zu verwenden. Für Tests ist **JUnit** zu nutzen.

Eine einfache Benutzeroberfläche ist erforderlich (z. B. HTML, Thymeleaf oder vergleichbar). Die Benutzeroberfläche muss funktional sein, jedoch kein komplexes Design enthalten.

Funktionale Anforderungen

Die Anwendung muss mindestens folgende Funktionalitäten unterstützen:

- Erstellung von Profilen für Tierhalter
- Erstellung von Profilen für Gastgeber

- Registrierung von Haustieren
- Erstellung einer Betreuungsanfrage (Zeitraum)
- Anzeige passender Angebote
- Versenden von Angeboten durch Gastgeber
- Annahme eines Angebots durch den Tierhalter
- Ablehnung weiterer Angebote
- Aktualisierung des Status einer Anfrage

Architektur

Die Anwendung muss klar strukturiert sein. Sie wählen zwischen einer klassischen **MVC- bzw. Schichtenarchitektur** oder einer einfachen **Microservice-Architektur** mit mindestens zwei Services.

Bei Verwendung von Microservices müssen die Verantwortlichkeiten klar getrennt sein. Die Kommunikation erfolgt über REST APIs. Die Architekturentscheidungen müssen nachvollziehbar begründet werden.

Testing

Testing ist ein verpflichtender Bestandteil der Projektarbeit.

Jede Gruppe muss mindestens **10 Unit Tests** implementieren. Für zusätzliche Integrationstests werden Bonuspunkte vergeben.

Alle Tests müssen sowohl im Code als auch in einer separaten Datei **TEST_DOCUMENTATION.md** dokumentiert werden. Für jeden Test ist zu beschreiben, was getestet wird, welches Ergebnis erwartet wird und ob es sich um einen normalen Fall oder einen Edge Case handelt.

Codequalität und Dokumentation

Der Code muss strukturiert, lesbar und nachvollziehbar sein. Wichtige Klassen und Methoden sind mit Kommentaren oder JavaDoc zu versehen. Geschäftslogik und Edge Cases müssen verständlich erklärt werden.

Security (Konzept)

Die Implementierung von Sicherheitsmechanismen ist **nicht verpflichtend**, wird jedoch mit Bonuspunkten bewertet, sofern sie sinnvoll umgesetzt und dokumentiert wird.

Verpflichtend ist ein **Security-Konzept**, das beschreibt, welche Daten im System sensibel sind, welche Risiken bestehen und welche Sicherheitsmaßnahmen in einem

realen Produkt berücksichtigt werden müssten. Außerdem soll erläutert werden, welche Maßnahmen bereits umgesetzt wurden (falls vorhanden) und welche in einer zukünftigen Version ergänzt werden sollten.

Zusätzlich ist das Konzept „**Shift Security Left**“ zu erklären und auf das eigene Projekt anzuwenden.

Einsatz von KI

Der Einsatz von KI-Tools ist erlaubt, muss jedoch vollständig dokumentiert werden.

Es ist anzugeben, welches Tool verwendet wurde, welches Modell bzw. welche Version eingesetzt wurde und für welche Aufgaben die KI genutzt wurde. Zusätzlich ist zu beschreiben, wie die generierten Ergebnisse überprüft, angepasst oder weiterentwickelt wurden und welche Teile eigenständig entstanden sind.

Falls KI verwendet wurde, muss im Repository eine Datei **KI_PROMPTS.md** enthalten sein. Diese Datei muss die verwendeten Prompts, deren Zweck sowie eventuelle Anpassungen oder Iterationen enthalten.

Alle Gruppenmitglieder müssen den gesamten Code verstehen und erklären können.

Zusätzlich ist zu reflektieren, welchen Einfluss KI auf Softwareentwicklung, Testing und Security hat und welche Risiken durch den Einsatz von KI entstehen können.

Entwicklungsprozess und Zusammenarbeit

Jede Gruppe muss ein Dokument erstellen, das beschreibt, wie die Zusammenarbeit organisiert wurde. Dazu gehört die Aufteilung der Aufgaben, der verwendete Entwicklungsansatz (z. B. Kanban oder Sprints), die Nutzung von Git sowie eine Reflexion über den Projektverlauf.

Die Beiträge aller Teammitglieder müssen klar erkennbar sein.

Erweiterungen und Kreativität

Zusätzliche Funktionalitäten über die Mindestanforderungen hinaus sind erlaubt und ausdrücklich erwünscht.

Alle zusätzlichen Anforderungen müssen klar dokumentiert und begründet werden. Für kreative und sinnvoll umgesetzte Erweiterungen werden Bonuspunkte vergeben.

Versionskontrolle und CI

Die Entwicklung erfolgt in einem gemeinsamen Repository (GitLab oder GitHub).

Erwartet werden regelmäßige Commits, die Nutzung von Branches sowie eine klare Nachvollziehbarkeit der Beiträge aller Teammitglieder.

Zusätzlich muss eine einfache CI-Pipeline vorhanden sein, die automatisch Tests bei Commits oder Pushes ausführt.

Abgabe

Abzugeben sind der vollständige Quellcode im Repository, eine Architekturbeschreibung, die Testdokumentation, das Security-Konzept, das Dokument zum Entwicklungsprozess sowie eine Präsentation mit Demo.

Während der Präsentation kann jedes Gruppenmitglied gebeten werden, Teile der Implementierung zu erklären.

Bewertung

Die Endnote setzt sich aus zwei Hauptteilen zusammen.

1. Präsentation (33%)

Die Präsentation wird anhand folgender Kriterien bewertet: Struktur und roter Faden, Fachwissen und kritische Reflexion, Visualisierung, Auftreten und Kommunikation sowie die Fähigkeit, Fragen zu beantworten und eine Diskussion zu führen.

2. Artefakte (67%)

Die Bewertung der Projektergebnisse erfolgt anhand folgender fünf gleich gewichteter Bereiche:

Code (20%)

Bewertet werden die funktionierende Umsetzung der Anforderungen, die Qualität und Struktur des Codes sowie dessen Lesbarkeit und Wartbarkeit.

Anforderungen und Architekturdokumentation (20%)

Bewertet werden die Klarheit der Anforderungen, die Nachvollziehbarkeit der Architektur sowie die Qualität der getroffenen Designentscheidungen.

Testing (20%)

Bewertet werden Umfang und Qualität der implementierten Tests, insbesondere die mindestens 10 Unit Tests, die Dokumentation der Tests sowie die Berücksichtigung von Edge Cases. Integrationstests werden positiv berücksichtigt.

Security-Verständnis (20%)

Bewertet werden das Security-Konzept, die Identifikation relevanter Risiken sowie das Verständnis des Prinzips „Shift Security Left“. Implementierte Sicherheitsmaßnahmen werden zusätzlich positiv bewertet.

Versionskontrolle, Teamarbeit und CI (20%)

Bewertet werden die Zusammenarbeit im Team, die sichtbaren Beiträge aller Mitglieder,

der sinnvolle Einsatz von Git sowie die automatische Ausführung von Tests über eine CI-Pipeline.

Wichtige Hinweise

Alle Gruppenmitglieder müssen aktiv beitragen. Der eingereichte Code muss vollständig verstanden und erklärbar sein. Der Einsatz von KI ist erlaubt, ersetzt jedoch nicht das Verständnis der Lösung. Kreativität wird extra belohnt, wenn im Kontext sinnvoll begründet ist.